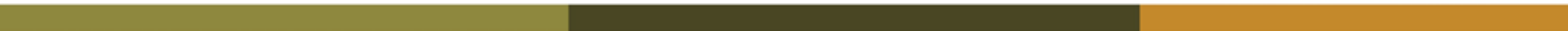


Randomized Algorithms

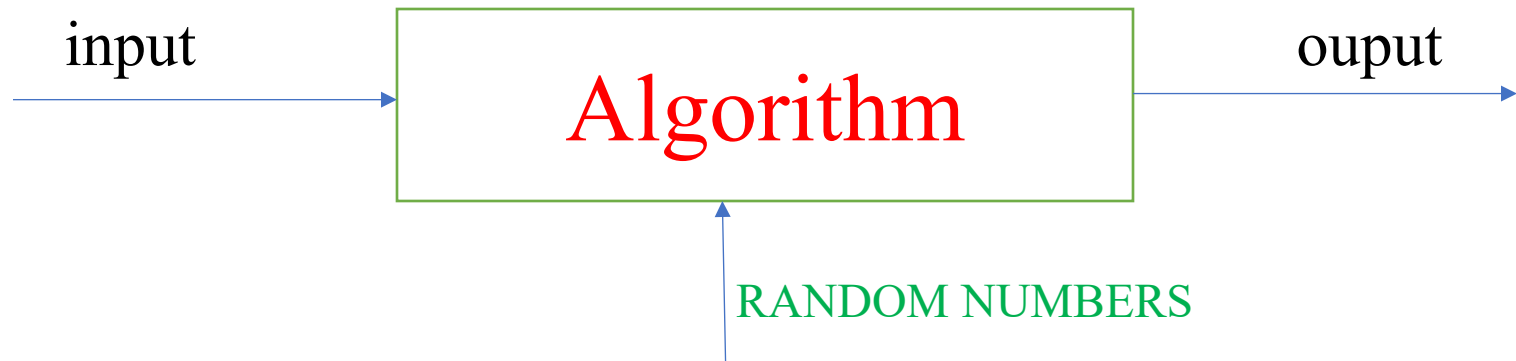


Deterministic Algorithms



- Goal to prove that the algorithm solves the problem correctly always and quickly. Typically, the number of steps should be polynomial in the size of the input.

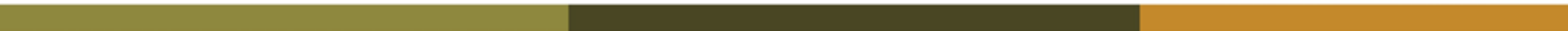
Randomized Algorithms



- In addition to input algorithm takes a source of random numbers and makes random choices during execution.
- Random algorithms make decisions on rolls of the dice.
- Example : Quick sort, Shuffling Cards, Round-Robin CPU Scheduling with random arrival of processes, etc.

Why use randomness?

- Avoid worst-case behavior: randomness can (probabilistically) guarantee average case behavior
- Efficient approximate solutions to inflexible problems.



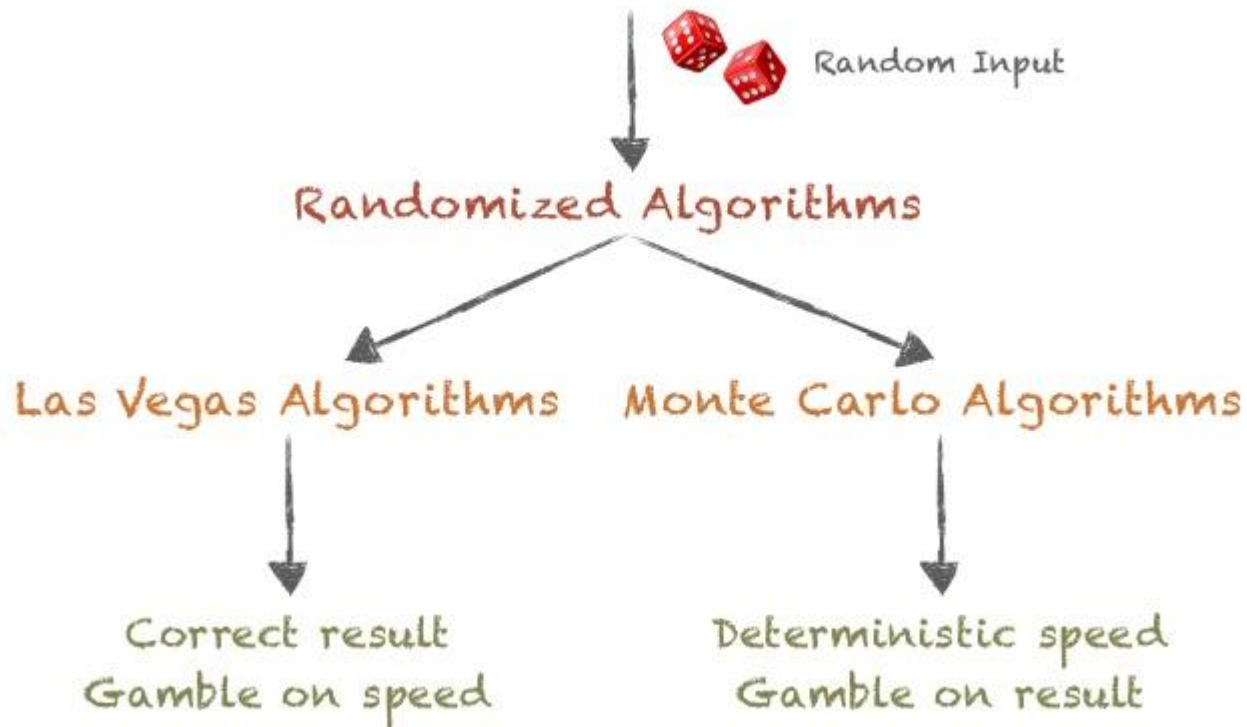
Randomized Algorithms

- A randomized algorithm is just one that depends on random numbers for its operation.
- These are randomized algorithms:
 - ◆ Using random numbers to help find a solution to a problem.
 - ◆ Using random numbers to improve a solution to a problem.
- These are related topics:
 - ◆ Getting or generating “random” numbers.
 - ◆ Generating random data for testing (or other) purposes.

Types of Random Algorithms

- Las Vegas:
 - Guaranteed to produce correct answer, but running time is bounded by input size.
- Monte Carlo:
 - Running time bounded by probability, and answer may not be appropriate.

Types of Random Algorithms



Las Vegas Algorithms

- Always gives the true answer.
- Running time is random.
- Running time is bounded.
- Randomized Quick sort is a Las Vegas algorithm.
- A Las Vegas algorithm always produces the correct answer its running time is a random variable whose expectation is bounded (say by a polynomial).

Monte Carlo Algorithms

- It may produce inappropriate result!
- We are able to bound its probability.
- By running it many times on independent random variables, we can make the failure probability arbitrarily small at the expense of running time.
- A Monte Carlo algorithm runs for a fixed number of steps and produces an answer that is correct with probability $< 1/2$.

Advantages of randomized algorithm

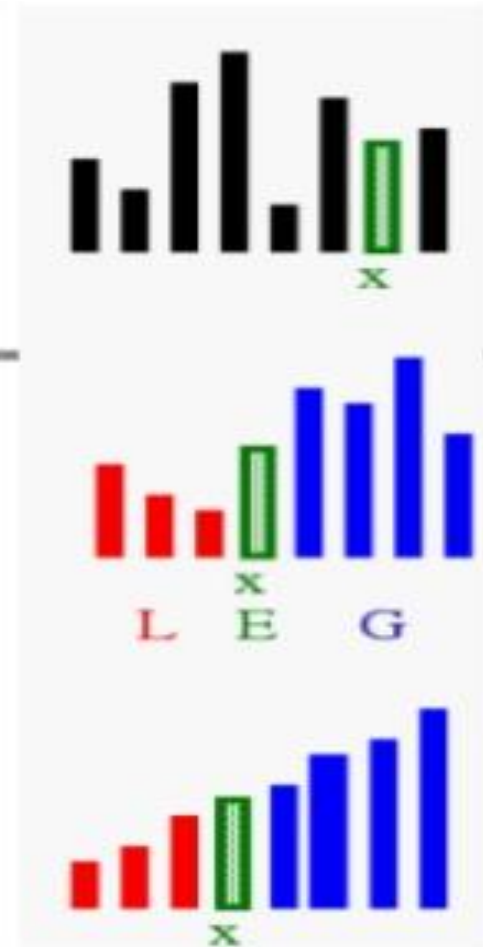
- Simplicity.
- Efficiency.
- Performance.
- For many problems, a randomized algorithm is the simplest, the fastest, or both.

Quick Sort

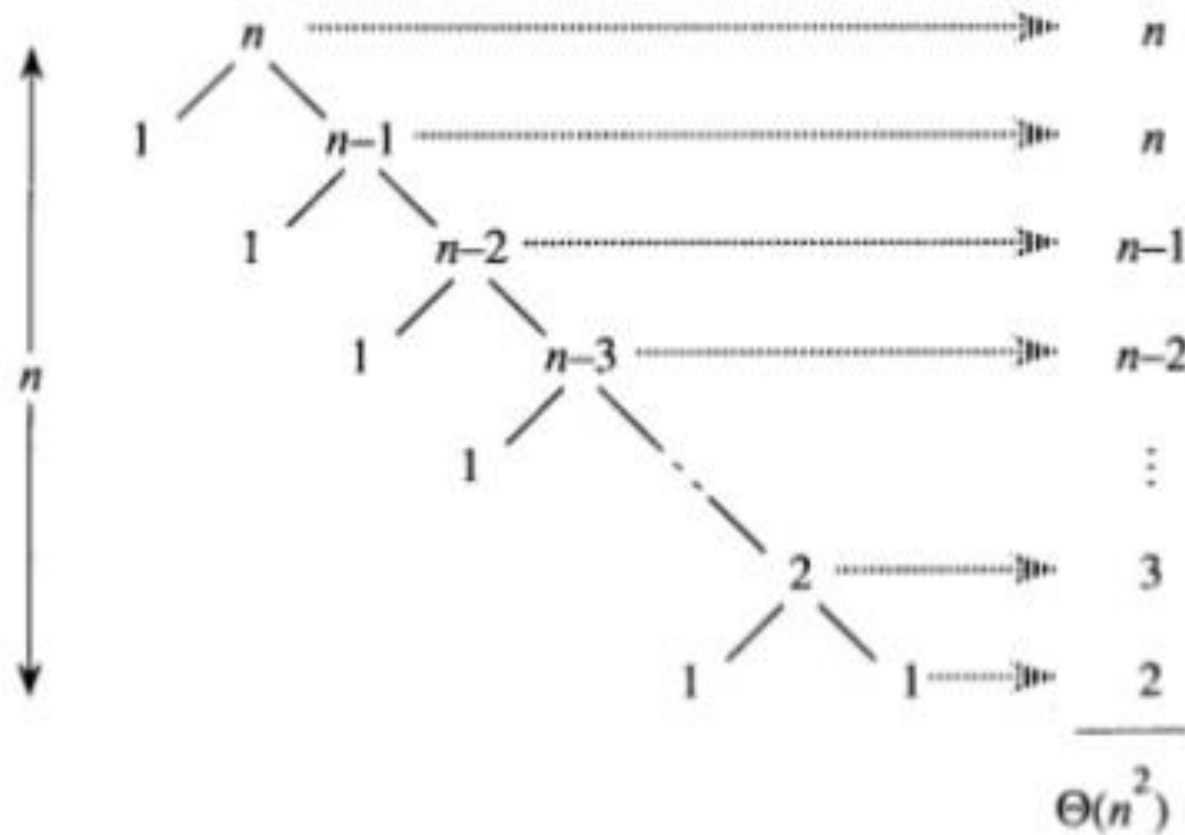
Select: pick an arbitrary element x in S to be the pivot.

Partition: rearrange elements so that elements with value less than x go to List L to the left of x and elements with value greater than x go to the List R to the right of x .

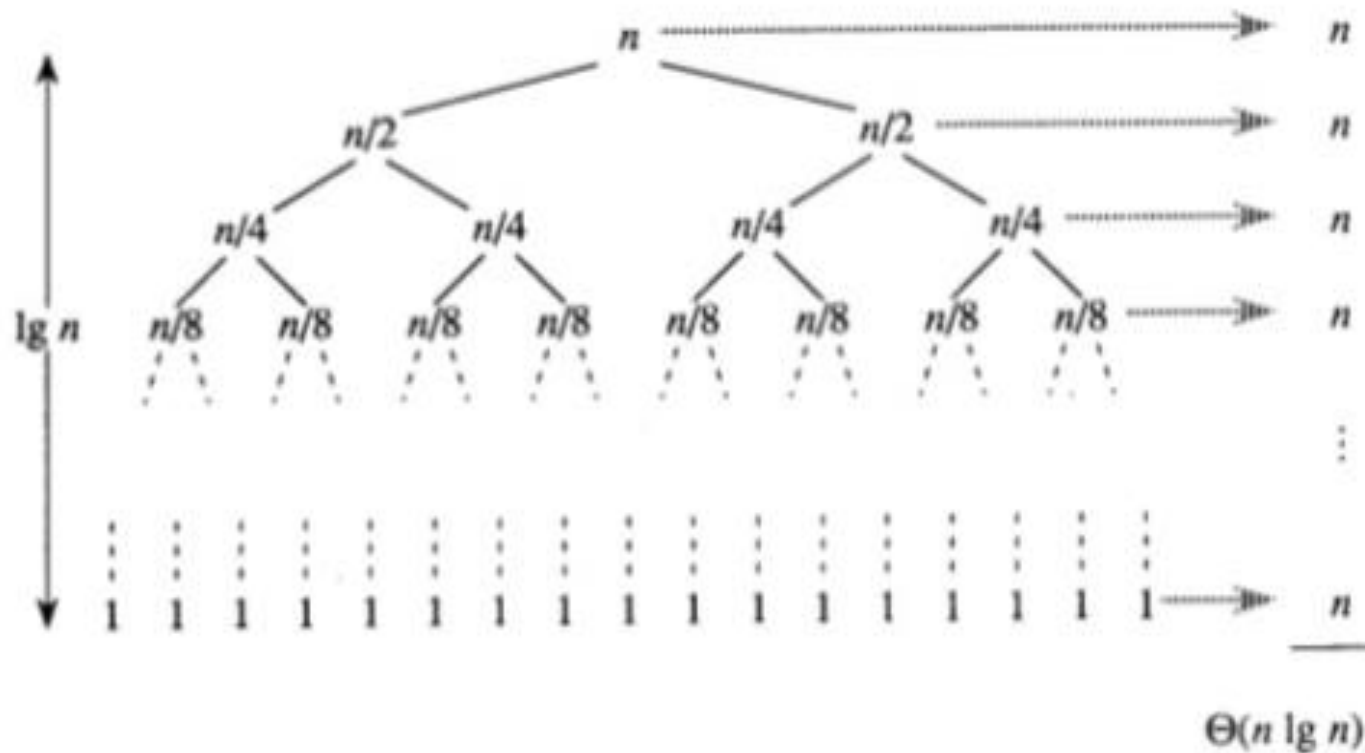
Recursion: recursively sort the lists L and R .



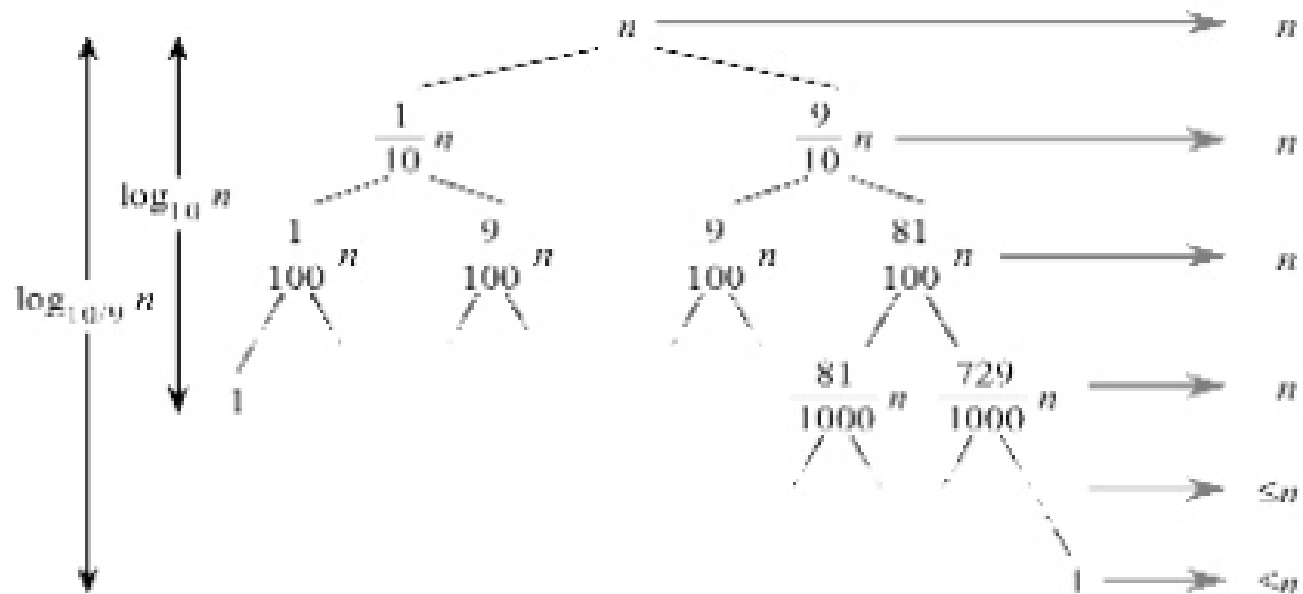
Worst Case Partitioning of Quick Sort



Best Case Partitioning of Quick Sort



Avg Case of Quick Sort



$$\Theta(n \lg n)$$

Randomized Quick Sort

Randomized-Partition(A, p, r)

1. $i \leftarrow \text{Random}(p, r)$
2. exchange $A[r] \leftrightarrow A[i]$
3. **return** Partition(A, p, r)

Randomized-Quicksort(A, p, r)

1. **if** $p < r$
2. **then** $q \leftarrow \text{Randomized-Partition}(A, p, r)$
3. **Randomized-Quicksort**($A, p, q-1$)
4. **Randomized-Quicksort**($A, q+1, r$)

PARTITION(A, p, r)

- 1 $x = A[r]$
- 2 $i = p - 1$
- 3 **for** $j = p$ **to** $r - 1$
- 4 **if** $A[j] \leq x$
- 5 $i = i + 1$
- 6 exchange $A[i]$ with $A[j]$
- 7 exchange $A[i + 1]$ with $A[r]$
- 8 **return** $i + 1$

Randomized Quick Sort- Analysis

- Expected runtime: $O(n \log n)$.
- Expected runtime is a good measure of the performance of randomized algorithms, often more informative than worst case runtimes
- In most cases, it outperforms the heap sort and merge sort.

Applications: Simple Randomization Algorithms in Playing Card Games

- A randomized algorithm is an algorithm whose behavior depends, in part, on the outcomes of random choices or the values of random bits.
- The main advantage of using randomization in algorithm design is that the results are often simple and efficient.
- In addition, there are some problems that need randomization for them to work effectively.
- For instance, consider the problem common in computer games involving playing cards—that of **randomly shuffling a deck of cards so that all possible orderings are equally likely.**

Overhand Shuffling



- Shuffling card is an essential part of every card game.
- There are many techniques for shuffling the cards.

Overhand shuffle

- A deck of cards is essentially a fixed sized array of length 52. Overhand shuffle puts set of cards from the end of the array to the beginning of an array.
- This process gets repeated to get a good shuffle.

Riffle Shuffling



Riffle shuffle

- This involves cutting the deck into 2 halves so we get two sets of cards and riffle them such that at the end both halves gets interleaved.

```
1 int i = mid;
2 int j = arr.length - 1;
3 while(i > 0){
4     int temp = arr[i];
5     arr[i--] = arr[j];
6     arr[j--] = temp;
7 }
```

Fisher-Yates Random Shuffling

- **Fisher-Yates** shuffle is one such algorithm for achieving a perfect shuffle using a random number generator.
- The algorithm is named after Ronald Fisher and Frank Yates who first described this algorithm in their book in 1938.
- Later Donal Knuth and Richard Durstenfeld introduced an improved version of the algorithm in 1964.

```
1 Random rnd = new Random();
2 for (int i = ar.length - 1; i > 0; i--)
3 {
4     int index = rnd.nextInt(i + 1);
5     // Simple swap
6     int a = ar[index];
7     ar[index] = ar[i];
8     ar[i] = a;
9 }
```

Fisher-Yates Random Shuffling

- Algorithm:
 1. First, fill the array with the values in order.
 2. Go through the array and exchange each element with the randomly chosen element in the range from itself to the end.

Fisher-Yates Random Shuffling

- `deck = ["Ace of hearts", "2 of hearts", "3 of hearts", "4 of hearts", "5 of hearts", "6 of hearts", "7 of hearts", "8 of hearts", "9 of hearts", "10 of hearts", "Jack of hearts", "Queen of hearts", "King of hearts", "Ace of diamonds", "2 of diamonds", "3 of diamonds", "4 of diamonds", "5 of diamonds", "6 of diamonds", "7 of diamonds", "8 of diamonds", "9 of diamonds", "10 of diamonds", "Jack of diamonds", "Queen of diamonds", "King of diamonds", "Ace of clubs", "2 of clubs", "3 of clubs", "4 of clubs", "5 of clubs", "6 of clubs", "7 of clubs", "8 of clubs", "9 of clubs", "10 of clubs", "Jack of clubs", "Queen of clubs", "King of clubs", "Ace of spades", "2 of spades", "3 of spades", "4 of spades", "5 of spades", "6 of spades", "7 of spades", "8 of spades", "9 of spades", "10 of spades", "Jack of spades", "Queen of spades", "King of spades"]`

